# Uplink: How It Works

One photo of leftover food becomes a safe, completed donation, decided by three AI agents with competing goals and no boss.

A complete guide to the project: what it does, how the agents work together, the architecture, and how to explain it to others.

UCSC AI Hackathon · Cisco Cloud AI track, "When AI Agents Disagree" · October 10, 2026


1. Uplink in one minute
2. The problem and who uses it
3. A donation's journey
4. System architecture
5. The agentic workflow
6. Safety rules and red lines
7. When the agents should not agree
8. The wider agent network
9. Reading the photo
10. Voices
11. The two websites
12. Evidence: does the team do better?
13. What's real and what's simulated
14. Code map
15. How to run it
16. Explaining Uplink to others
17. Glossary

## 1 Uplink in one minute

Uplink turns one photo of leftover food into a safe, completed donation. A business photographs its surplus food, AI identifies it, and three AI agents with competing goals work out where it should go and who should drive it.

| AGENT | WHAT IT WANTS | A TYPICAL ARGUMENT |
|---|---|---|
| **Speed** | Food delivered fast and reliably, long before it spoils | "Hot food can't wait. The paid courier is 8 minutes away." |
| **Cost** | Spend as little as possible on drivers | "Use the volunteer. She's free." |
| **Need** | Food goes to whoever has waited longest | "St. Francis Catholic Kitchen hasn't had a donation in 4 days." |

No agent is the boss. They propose plans, vote, and give ground until they agree. **Safety rules written in plain code** remove every unsafe option before the debate starts, so the agents can never argue their way into an unsafe delivery. When they truly can't agree, or agreeing would cross a line one of them refuses to cross, they **stop and ask a person** instead of forcing a bad decision.

Every agent runs as **its own service**: shelters, drivers and the three negotiators find each other through an agent directory and talk over the network, like a small marketplace. When the food arrives, Uplink writes the donation record that California's SB 1383 law requires.

> **THE ONE-SENTENCE PITCH**
> Food rescue is full of real tradeoffs. Our agents negotiate them with no boss, safety is never up for debate, and when they truly can't agree, they ask a human.

---

## 2 The problem and who uses it

California's SB 1383 requires many food businesses (grocery stores, large restaurants, hotels, caterers, schools) to donate edible surplus food and keep records of it. Much of it is still thrown away, because donating is a hassle: someone has to find a shelter that can take *this* food *tonight*, check it has a fridge, find a driver, and do the paperwork.

Every one of those choices is a tradeoff. The fastest driver costs money. The free volunteer may not show up. The nearest shelter may already have been fed today, while another has waited four days. Uplink cuts the business's part down to one photo and lets agents handle the tradeoffs.

| WHO | WHAT THEY WANT | WHAT THEY USE IN UPLINK |
|---|---|---|
| **Business**<br>café, bakery, hotel kitchen | Get leftover food picked up now, with proof for compliance | The customer site at `/`: schedule a pickup, track it, download the receipt |
| **Shelter or pantry** | Food it can actually use, with no surprises | The shelter phone page at `/shelter`: accept, take part, or decline each request, by tap or by voice |
| **Driver**<br>volunteer, paid courier, a nonprofit's own van | Trips that fit their range and schedule | Simulated in this demo; each driver has its own agent that bids on trips |
| **Coordinator**<br>the human in the loop | Step in only when the agents shouldn't decide alone | The Decisions inbox on the ops dashboard |
| **Judges, presenters** | See how the agents think, and proof it works | The ops dashboard at `/ops`: live negotiation, map, logs, evidence, how it works |

**WHY IT'S DIFFERENT FROM EXISTING APPS**

Services like Copia already match donations to nonprofits, but businesses still have to enter details and coordinate. Uplink cuts it to one photo, and the agents handle matching, failures and paperwork, including the hard tradeoffs a fixed rule gets wrong.

## 3 A donation's journey, step by step

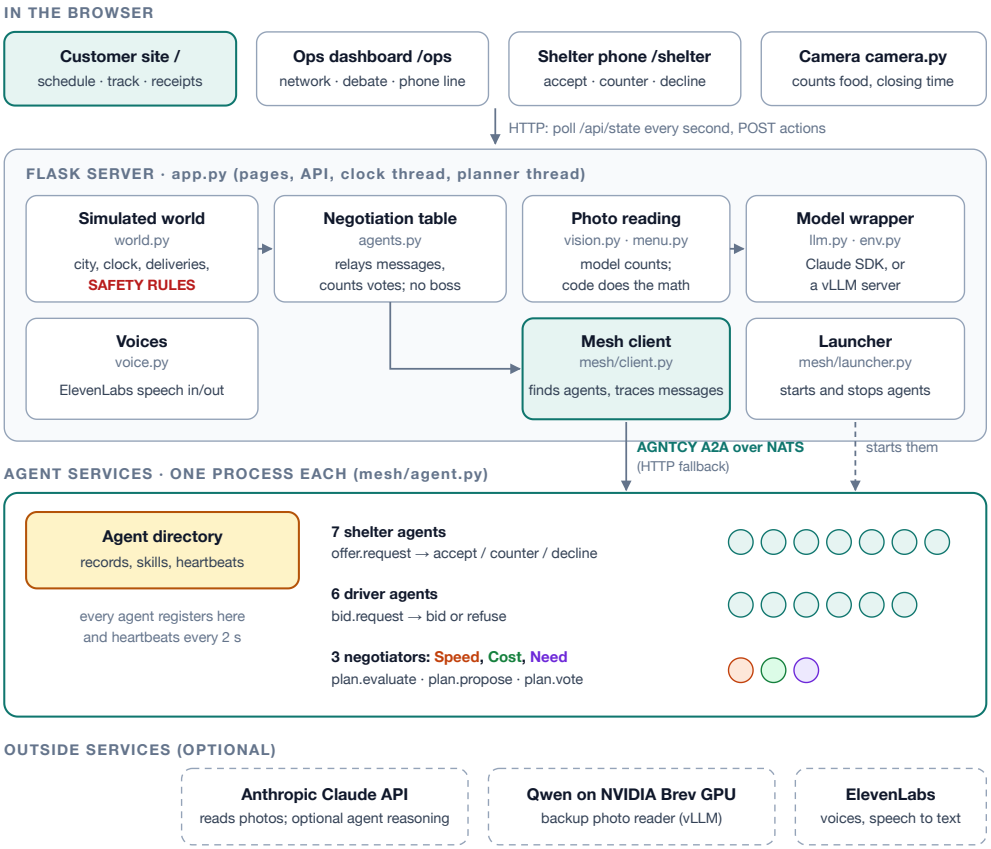
At 2:00 PM India Gourmet, in Abbott Square Market, has 30 servings of chicken tikka masala left over. Here is everything that happens next, in plain words, with the technical name of each step and the file that does it.

1. **The manager takes a photo** (or a short video, or just says what's left) on the customer site. (vision · `vision.py`)
   The photo goes to an AI vision model, which names the food, counts it, and says how sure it is. Uplink's own code, not the AI, turns the count into servings, weight and how long the food stays safe.
2. **The manager checks and confirms.** If the AI got something wrong, they fix it. If they aren't sure about allergens, they tick "Not sure".
3. **The food is announced to the network.** (agent network · `network.py`)
   Every shelter's agent answers from its own point of view: "Yes, we can take all 30", "We can take 12 of the 30", or "Sorry, we're full". Every driver's agent bids on each trip with an arrival time and a price, or says no if it's too far.
4. **Safety rules remove anything unsafe.** (safety rules · `world.py`)
   No fridge for cold food, an allergen the shelter must avoid, closed by arrival time, no room, or arriving too close to when the food spoils. Those options disappear before any agent sees them.

5. **Speed, Cost and Need negotiate.** (negotiation · `agents.py`)
   Each proposes a plan, votes on every plan on the table, and gives ground over up to three rounds. Here they agree: Beach Flats Community Center, via a paid courier arriving in about 10 minutes.
6. **If they can't or shouldn't agree, a person decides.** (escalation)
   The question lands in the coordinator's Decisions inbox with the reason, the agents' top two options, and a "decide by" time.
7. **The shelter confirms.** (shelter phone)
   If someone is answering the shelter's phone page, they accept, take part, or decline. Otherwise the plan goes ahead.
8. **The driver picks up and delivers.** (simulated world)
   The manager watches on the tracking page: "Driver on the way, arrives 2:19 PM." (times here are an example). Its "Updates" list says what happened in plain words; the agents' debate stays on the ops dashboard.
9. **Something may change mid-route.** (adaptation)
   A shelter fills up, or a volunteer cancels. The agents re-plan from wherever the food is now, and each says whether the news changed its pick.
10. **Delivered, and the record is written.** (SB 1383 record)
    Donor, recipient, items, weight, pickup and delivery times and transport type are logged automatically, ready to print or download.

## 4 System architecture

Uplink is a Python web server (Flask) that simulates the city, plus **one separate process per agent**. Each agent is also an **AGNTCY A2A agent over NATS** (Cisco's open agent stack): the agents find each other through an agent directory and exchange A2A messages, one broadcast per step. The agents' decisions are real; the city around them (shelters' locations, drivers moving, the clock) is simulated.




Each agent is a separate process that registers in the directory and answers messages. Dashed boxes are optional; without any keys everything still runs.

**IN PLAIN WORDS**

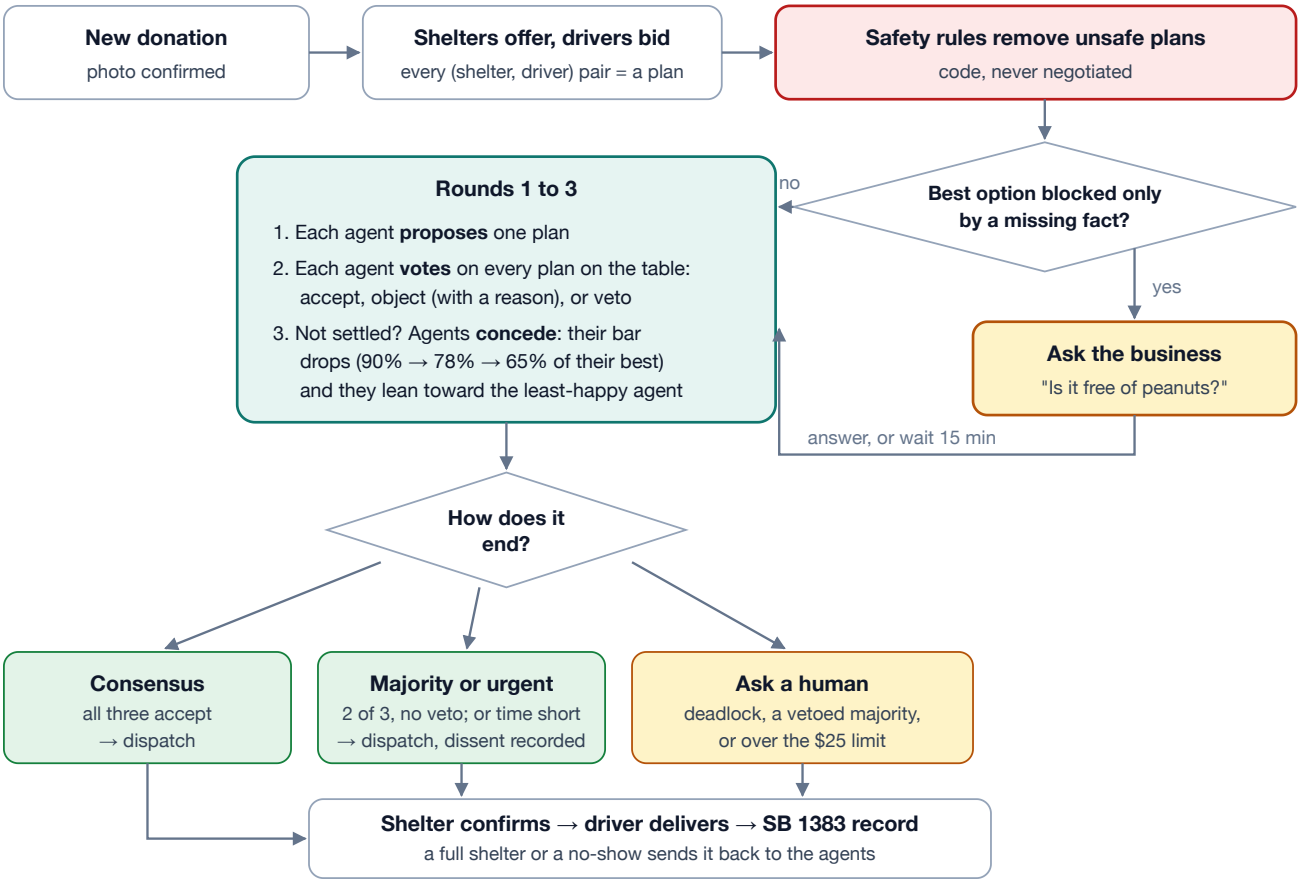
Each shelter, driver and negotiator is its own little program, like separate people with their own phones. A directory works like a phone book: agents list themselves there, and anyone can look up who can take food or drive a trip. If one agent goes offline, it drops out of the phone book and the others carry on.

**TECHNICAL DETAIL**

The **clock thread** advances simulated time (1 simulated minute per real second at 1×). The **planner thread** runs each negotiation outside the shared lock, so a slow agent never freezes the dashboard. Each agent process serves an A2A AgentCard over NATS with the AGNTCY app-sdk (and an HTTP fallback); the launcher starts about 16 of them, plus the NATS server if needed, and they all exit when the app exits.

## 5 The agentic workflow: negotiation with no boss

Three agents with competing goals decide every plan together. Shared code only counts the votes; no supervisor picks the winner.




One donation's decision. Every step between agents is a real message: offers, bids, and each negotiator's evaluation, proposal and votes. The agents never see an unsafe plan, and a human is asked only when agreeing would be wrong.

**IN PLAIN WORDS**

Think of three coworkers with different priorities picking a delivery plan. Each suggests their favourite, everyone gives a thumbs up or down with a reason, and if nobody budges, each is willing to accept a slightly worse option for their goal. Each also has one line they will never cross.

**TECHNICAL DETAIL**

Each agent scores every safe plan from 0 to 1 against its own goal. It accepts a plan scoring at least 90%, 78% then 65% of its best option in rounds 1, 2 and 3. When proposing in later rounds it blends its own score with the least-happy other agent's (weight 0, 0.45, 0.75). Vote weights start at 1.0; a majority needs 2.0. Before round 1, each negotiator sends the others its scores and vetoes for every plan (`plan.evaluate`), so every agent argues from shared evidence.

### Each agent argues in its own terms

Even when agreeing, each says why in the language of its goal. Speed: "arrives with 34 min to spare and Rapid Courier A is 95% reliable." Cost: "costs nothing." Need: "Beach Flats hasn't had food in 4 days. Any safe driver is fine with me", or "Accept, reluctantly" when the shelter isn't the neediest. Need judges *which shelter*, not which driver; in 60 random runs its first pick became the decision 232 of 334 times.

## How each agent scores a plan

| AGENT | SCORE (0 TO 1) | RED LINE: ALWAYS VETOES |
|---|---|---|
| **Speed** | Minutes to spare ÷ minutes left, weighted by driver reliability | Under 10 minutes to spare, or a driver under 50% reliable |
| **Cost** | Cheaper is better, weighted by reliability (a no-show means paying a courier to rescue the food) | Over $20 for one donation |
| **Need** | Days since the shelter's last donation (70%) and room left (30%) | Feeding a shelter that got food today while another has waited 4+ days |

**RULES OR AI REASONING**

By default the agents reason with these scoring rules, which are fast and repeatable for demos and tests. With `LLM_AGENTS=1` and a Claude key, each agent's proposals, votes and reasons come from Claude, told its goal and red line. Each Claude agent sees only the facts its own goal is about, and votes against the same per-round bar as the rules; accepting far below that bar is overruled by its rules vote, so it can't just give in. Red lines are still enforced in code, and any failed AI call falls back to the rules for that agent. The Evidence runs always use the rules: fast, free and repeatable.

## 6 Safety rules and red lines

There are two kinds of limits. Safety rules protect people and are never up for debate. Red lines protect each agent's goal and act as vetoes inside the debate.

### Safety rules: plain code, applied before any agent sees a plan

| RULE | A PLAN IS REMOVED WHEN… |
|---|---|
| Food type | The shelter doesn't accept that category (hot prepared, cold prepared, packaged, produce) |
| Cold chain | The food must stay cold and the shelter has no fridge |
| Allergens | The food contains an allergen the shelter must avoid, or allergens are unconfirmed and the shelter avoids any (unconfirmed counts as present) |
| Capacity | The shelter has no room, counting food already on its way |
| Opening hours | The shelter would be closed when the driver arrives |
| Safe time | The food would arrive less than 5 minutes before it becomes unsafe |
| Full | The shelter has said it's full |

> **WHY THIS MATTERS**
> "Is the AI deciding if food is safe?" No. Safety is fixed rules in code. The agents only choose among options that already passed them, so they can never pick an unsafe one, however persuasive an argument is.

### Red lines: each agent's veto

A veto holds in every round, and a majority cannot override it. If the only plan two agents like is vetoed by the third, the agents don't force it through: they hand the choice to a person and explain why. That is how Uplink shows **"consensus is not always the right outcome"**, which the Cisco challenge asks for. The red lines are in the scoring table in section 5.

## 7 When the agents should not agree

A negotiation can end six ways. Three act on a plan, and three stop and ask a person, because agreeing would be wrong.

| OUTCOME | WHEN IT HAPPENS | WHAT HAPPENS NEXT |
|---|---|---|
| **Consensus** | All three agents accept a plan | It goes ahead |
| **Majority with dissent** | After 3 rounds, 2 of 3 (by vote weight) accept a plan nobody vetoed | It goes ahead; the losing agent's objection is kept on record |
| **Time is short** | Under 30 minutes until the food is unsafe | Stop debating; act on the best-supported plan nobody vetoed |
| **Unacceptable compromise** | The majority's plan crosses the third agent's red line | Ask a human, with the reason and the top two options |
| **Irreconcilable goals** | After 3 rounds, no plan has majority support | Ask a human |
| **Insufficient evidence** | An agent's best plan is blocked only by an unconfirmed fact (allergens), with 45+ minutes left | Ask the business; wait up to 15 minutes, then act on what is known |

A plan that costs more than $25 also goes to a human.

### Fallback rules (disclosed, not decision-makers)

- **Nobody answers the coordinator.** The agents act at the last moment an option can still arrive safely, using their best-ranked option that still makes it. The Decisions card shows this "decide by" time.
- **Any approval is re-checked.** If time has passed and an option would now arrive too late, it isn't dispatched; the agents find one that still works.
- **Scripted test runs** have no human at the keyboard, so a simulated coordinator picks the agents' top-ranked option. The activity feed labels all of these.

> **WHY THE FALLBACKS WERE REDESIGNED**
>
> The first version waited until 15 minutes before the food spoiled. In testing, every option by then took 25 minutes, so food arrived spoiled. Now the deadline is computed from the options themselves (the latest safe departure, minus 2 minutes), so the fallback always leaves a plan that still works.

## 8 The wider agent network

Beyond the three negotiators, every shelter and every driver has its own agent. Together they behave like a small marketplace with no central dispatcher.

**Business agent**

Announces the surplus to the network: what it is, how much, and how long it's safe.

**Shelter agents**

Each answers from its own situation: accept all, counter-offer to take only part (for example "12 of 30, we're low on room"), or decline with a reason.

**Driver agents**

Each bids on each trip with an arrival time, a price and its reliability, or refuses a trip beyond its range (volunteers drive 12 km at most by default).

**Speed, Cost, Need**

Negotiate over every (shelter offer × driver bid) combination that passed the safety rules.

### What makes the network adaptive

| BEHAVIOUR | HOW IT WORKS |
|---|---|
| **Discovery** | Agents search a directory of shelters by capability (food types, fridge, hours, room). A new pantry can join mid-run, and waiting donations are reconsidered. |
| **Re-planning** | A shelter saying "full" or a driver cancelling sends the donation back to the agents. If the driver already has the food, they re-plan from the driver's position. |
| **The effect of new information** | A re-planned debate opens with "New information: ..." (a cancellation, a full shelter, a new pantry, the business confirming allergens), and each agent says whether it changed its pick: "That changes my pick: I wanted X, now Y" or "Still X; this doesn't change my pick." |
| **Learning who was right** | A driver who cancels loses 30 points of reliability, and every agent sees it. An agent that warned about that driver gains +0.25 vote weight for the rest of the run. |
| **Split donations** | If the winning plan is a counter-offer, the rest of the food is split off and announced again, so nothing is left behind. |
| **Combined trips** | Two donations from the same pickup can ride in one vehicle (up to 150 servings), keeping another driver free. Cost values this. |
| **Shelter phones** | When a person is answering a shelter's phone page, the agents ask first and hold the driver for up to 8 simulated minutes, then try elsewhere. |

### Every agent is its own service

| PIECE | HOW IT WORKS |
|---|---|
| **Agent directory** | Each agent publishes a record: its id, name, skills (for example `food.receive`, `trip.bid`, `plan.vote`), what it can do (food types, fridge, range) and its address. Modeled on AGNTCY's Agent Directory and OASF agent records. |
| **Heartbeats** | Every agent checks in every 2 seconds. One that goes silent for 6 seconds, or shuts down, drops out and is no longer discovered. |
| **Messages** | **AGNTCY A2A over NATS**, one broadcast per step: `offer.request` to every shelter, `bid.batch` to every driver, `plan.evaluate`, `plan.propose` and `plan.vote` to the three negotiators. HTTP is the automatic fallback. Every message is traced with sender, receiver, type and reply time (a few ms locally). |
| **Resilience** | Stop a shelter's agent and it isn't consulted; the food goes elsewhere. A pantry or driver that joins mid-run gets its own agent, which registers itself. If a negotiator is down, the same logic stands in for it and that is logged. |
| **Proof it's the same** | `tests/test_mesh.py` replays all 9 scenarios through the real processes twice, over AGNTCY A2A/NATS (about 1,100 messages, all over A2A) and over HTTP (about 1,700), with 0 failures, and checks every number and every negotiation matches the in-process version exactly. |

> **WHAT'S AGNTCY AND WHAT'S OURS**
>
> From AGNTCY: the A2A protocol, the AgentCards, the NATS transport and the broadcast pattern (via the AGNTCY app-sdk, as in Cisco's coffeeAGNTCY demo), and OASF records. Ours: the agents and their decisions, the safety rules, the negotiation, and the agent directory with heartbeats (modeled on AGNTCY's Agent Directory).

---

## 9 Reading the photo

The AI model only does what it's good at: naming the food and counting what it can see. Uplink's code does the math, so the same count always gives the same numbers.

1. **Prepare the photo.** Fix its rotation, shrink it to 1568 pixels on the long side, and convert iPhone HEIC photos to JPEG. This happens in the browser and again on the server.
2. **The model identifies and counts.** For each kind of food it gives a count range (a range when some is hidden), says how it counted ("3 rows of 4"), lists likely allergens, and states its confidence: high, medium or low.
3. **Code computes the rest** from a unit table: 1 bagel ≈ 1 serving and 0.25 lb, 1 pizza slice ≈ 1 serving, 1 catering tray ≈ 10 servings. Safe time comes from the category (hot prepared about 2 hours, cold about

90 minutes, packaged 24 hours).

4. **The business checks.** Low-confidence counts are highlighted, and every field can be edited before confirming.
5. **Never a fake answer.** If the call fails or the photo isn't food, the form comes back empty with the reason. With no model connected, a clearly labeled demo result appears instead.
6. **A rejected key is caught.** At startup one cheap call checks the Claude key. If Claude rejects it, Uplink stops retrying, behaves as if there were no key (rules agents, demo photo result) and says which key it was, showing only its first and last characters.

**Or send a short video.** Pan across the food. The browser takes four frames spread across the clip, tiles them into one image and sends that; the model is told these are four views of the same food, so it counts each item once. Nothing extra is needed on the server. (Some iPhone .mov files won't decode in Chrome; MP4 or a photo works.)

**Or just say it.** On the "Say it" tab, tap the mic and say what's left: "two dozen bagels and 4 country loaves." ElevenLabs Scribe writes it down, Claude matches it to the business's menu in menu units, and items not on the menu are offered for adding. Without Claude, word matching handles amounts ("two dozen"), plurals and near-misses, and won't guess between two similar items. Without ElevenLabs, the browser's own speech recognition is used.

**Or pick from your menu.** Each business has its own menu. Instead of a photo or typing, staff can pick leftovers straight from it, and after a photo the items are matched to the menu so allergens and category come from the business's own data.

### Two ways to run the model

| OPTION | HOW | TRADEOFFS |
|---|---|---|
| **Claude** (current) | Anthropic's API, through the official Python SDK. Key in `.env`. | Most accurate; about 2 to 4 cents per photo; needs internet |
| **Qwen2.5-VL on an NVIDIA Brev GPU** (backup) | An open model served by vLLM on a rented L4 GPU, reached through a private tunnel. `scripts/start_vision.sh` | No API key needed; less accurate; about $0.90 per hour while running |

> **TESTED RESULT**
> A photo of a cut vegetable pizza came back as "vegetable pizza, 8 slices, high confidence", noting one slice had a bite taken out. A busy bagel display came back as 23 labelled kinds, 240 to 367 bagels, low confidence, which is honest because many were stacked out of sight.

### The camera at the business

`camera.py` watches the counter with a fast on-device detector (YOLO-World) that counts food live. At closing time it sends one snapshot to the vision model, which splits the leftovers into one donation per food category, since safety rules differ by category. Video never leaves the laptop.

---

## 10 Voices

With an ElevenLabs key, the debate can be heard, and shelters can answer by speaking.

- **The agents speak.** Turn on Demo controls → Sound and look → Voices on the ops dashboard. Speed, Cost and Need each read their lines in a different voice, and a dispatcher voice announces the outcome. Each line waits for the previous one to finish.
- **The shelter hears requests.** On the shelter phone page, turn Voice on, and each new request is read aloud.
- **The shelter answers out loud.** Tap "Answer by voice" and say "yes", "no", or a number like "just 20". ElevenLabs turns speech into text, and Uplink's code turns that into accept, decline or take part.
- **The Phone line.** On the ops dashboard, after the agents choose a shelter, a dispatcher voice "calls" it: it reads out the food, driver and arrival time, and the shelter answers with its agent's actual reply ("We'll take 20 of the 30"). Voices use ElevenLabs Flash; spoken answers are transcribed with ElevenLabs Scribe. It's a voiced replay in the browser, not a call over the phone network; a staff member's real spoken answer from the shelter phone page appears in it too.
- **No key?** It falls back to the browser's built-in voice and speech recognition.

---

## 11 The two websites

Two audiences, two sites. The customer site shows what a real business would use. The ops dashboard shows judges what happens behind the scenes.

### Customer site · `/`

Built for a busy café manager who isn't technical. No agent jargon anywhere, only outcomes.

| PAGE | WHAT IT SHOWS |
|---|---|
| Home | "Leftover food? We'll get it picked up." One button: Schedule a pickup. |
| Schedule | 3 steps: photo or video, say it, pick from your menu, or type it in → "Does this look right?" summary → scheduled |
| Track | A status timeline (Received → Finding a place → Driver assigned → Picked up → Delivered), the shelter and arrival time in big text, and a map of only this pickup. If Uplink needs an answer, a simple yes/no question appears here. "Updates" explains in plain words what happened (no agent debate). |
| My donations | Past donations and printable SB 1383 receipts, with CSV download |
| Help | Short answers: what food is accepted, is it safe, what it costs (free), what record you get |

### Ops dashboard · `/ops`

| PAGE | WHAT IT SHOWS |
|---|---|
| Live | A dark judges' view: headline numbers; a live **agent network diagram** (business → shelter agents → driver agents → the three negotiators, with every message in a log beside it); the **Phone line**; the current donation; the three agents' positions and the outcome; and the city map. "See the debate" opens the full transcript and vote matrix. |
| Decisions | The coordinator's inbox: escalations with options and a "decide by" time. A red badge in the nav shows the count. |
| Activity | Agent network messages, agent feed, safety decision log, donations list, and **Agent services**: every agent's process, port, skills, message count and reply time, with Stop and Restart buttons and a live message trace |
| Evidence | Team versus baselines across scripted and randomized scenarios |
| Records · How it works | Printable SB 1383 records; a short visual explanation for judges |
| Demo controls | Scenario picker, speed, pause, voices, and "inject a problem" buttons: shelter full, driver cancels, new pantry joins, camera closing time |

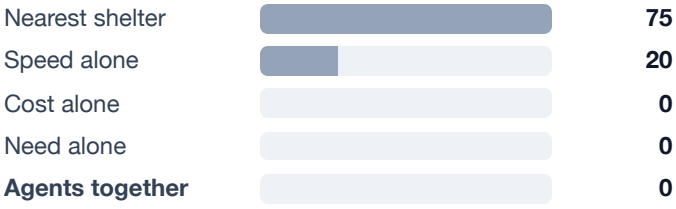
The shelter phone page ( `/shelter` ) is a third, phone-sized page a teammate can hold on stage to play a shelter.

## 12 Evidence: does the team do better?

The Cisco challenge asks: "Can independent AI agents make better decisions together than they can individually?" Uplink answers with numbers from repeatable scenarios.

The same 9 scripted scenarios run under five decision policies. **Nearest shelter** uses no agents and no safety checks. **Speed, Cost or Need alone** use the same safety rules as the team, but one agent decides by itself.

**Servings lost (lower is better)**



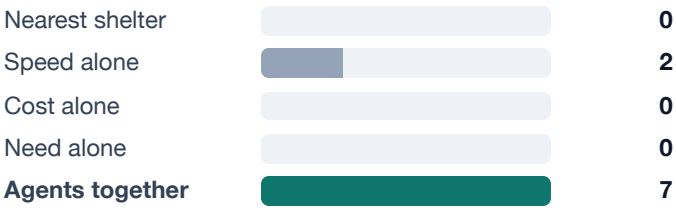

**Shelters fed (higher is better)**




**Money spent on drivers (lower is better)**




**Times a human was asked**




All 9 scripted scenarios combined, rules mode, from `python scenarios.py` on October 11, 2026. Meals delivered safely: 859 for nearest shelter, 914 for Speed alone, 934 for the rest.

### What the numbers say (honestly)

1. **The safety rules matter most.** Nearest shelter lost 75 servings; with the safety rules, only Speed alone lost any (20).
2. **The team spreads food most fairly.** It fed 23 shelters, more than any single agent.
3. **The team is not the cheapest.** It spent $189, more than Cost alone ($93), because it pays for reliable couriers and fairness. That is the tradeoff, and we say so.
4. **Refusing bad compromises involves people.** The team asked a human 7 times, where single agents mostly just acted. That is by design: those are the cases where agreeing would have been wrong.
5. **Real disagreement needs red lines.** Before red lines were added, every negotiation in the original scenarios ended in full agreement. Now the team's 53 decisions ended: consensus 42, majority 5, unacceptable compromise 3, insufficient evidence 2, irreconcilable 1.

### The strongest evidence: random scenarios the system had never seen

100 random scenarios (3 to 6 donations each, driver no-shows, full shelters, new pantries), not used while building:

| POLICY | SERVINGS LOST | MEALS DELIVERED | SHELTERS FED | DRIVER COST |
|---|---|---|---|---|
| Nearest shelter | 2,415 | 15,039 | 292 | $719 |
| Speed alone | 494 | 16,960 | 325 | $2,298 |
| Cost alone | 610 | 16,844 | 315 | **$374** |
| Need alone | 843 | 16,611 | 407 | $1,154 |
| **Team** | **164** | **17,290** | **459** | $704 |

**IN PLAIN WORDS**

Working together, the agents lost **a third as much food as the best agent working alone** (67% less) and 93% less than the simple approach, delivered the most meals and reached the most shelters. They lost the least food, or tied, in 99 of 100 runs. The trade-off: the cost-minimizing agent alone is cheaper.

## 13 What's real and what's simulated

The agents and their decisions are real. The world around them is simulated, which judges

expect at a hackathon.

**Real**

• The places: Santa Cruz businesses with items from their own menus, and Santa Cruz shelters and food programs at their real addresses (`data/SOURCES.md`)
• Photo and video reading by a vision model (Claude, or Qwen on a GPU)
• The agent loop: offers, bids, proposals, votes, red lines, outcomes
• Re-planning when a shelter declines or a driver cancels
• Safety rules and fallback rules
• Human approvals, business answers, shelter replies (tap or voice)
• Generated donation records
• Voices, speech recognition and spoken donations (with keys)
• Each agent as its own process, the agent directory, and the messages between them

**Simulated**

• Shelters' and food programs' capacity, hours, fridges and needs (the places themselves are real Santa Cruz organizations at their real addresses)
• Drivers: fictional people
• The clock: 1 simulated minute = 1 real second at 1× speed
• Driver movement along straight lines
• Events like a full shelter, a no-show or a new pantry, triggered from Demo controls or scripted scenarios
• The Phone line is voiced in the browser, not a real phone call (the shelter's words are its agent's real reply)

Skipped on purpose: logins and accounts, payments, real traffic routing, and training our own model.

## 14 Code map

| FILE | WHAT IT DOES |
|---|---|
| `app.py` | Flask server: page routes, the JSON API, the clock thread and the planner thread |
| `world.py` | The simulated city: directory, safety rules, drivers, clock, events, routes, records, fallback rules, the baseline and single-agent policies |
| `network.py` | Shelter agents (accept, counter, decline) and driver agents (bid or refuse) |
| `agents.py` | Speed, Cost and Need: scoring, red lines, the negotiation protocol, the evidence check, optional Claude reasoning |
| `mesh/` | The agent mesh: directory, one service per agent, the client that discovers and messages them (with a trace), and the launcher |
| `config.py` | Every tuning knob in one place: thresholds, timeouts, limits, speeds |
| `vision.py` | Photo → food details: preparation, the counting prompt, the unit table, failure handling |
| `llm.py`, `env.py` | Model wrapper (Claude SDK or an OpenAI-style server) and loading keys from `.env` |
| `menu.py` | Each business's menu: pick leftovers from it, and match photo results to it |
| `voice.py` | ElevenLabs text-to-speech and speech-to-text; understanding "yes", "no" and numbers |
| `camera.py` | The business camera: live food counting, closing-time snapshot |
| `scenarios.py` | 9 scripted scenarios and the five-policy comparison |
| `static/` | Web pages: customer site (`app.*`), ops dashboard (`dashboard.*`), shelter phone, records, shared styles |
| `data/` | Real Santa Cruz shelters, food programs and businesses with their real menus (simulated capacities and needs), fictional drivers; sources in `data/SOURCES.md` |
| `tests/` | `test_core.py`: 39 automated checks, such as cold food never going without a fridge. `test_mesh.py`: the agent processes decide identically and survive a stopped agent |
| `scripts/` | Brev GPU setup, start and stop scripts; the photo accuracy test |
| `docs/` | Architecture, results, Brev setup guide, and this document |

## 15 How to run it

> **START THE APP**
>
> `cd ~/uplink`
> `source .venv/bin/activate`
> `pip install -r requirements.txt` (first time only)
> `pip install -r requirements-agntcy.txt` and `brew install nats-server` (first time only, for AGNTCY A2A over NATS; without them the agents talk over HTTP)
> `python app.py` (port 5050 by default; it also starts the agent processes)

| PAGE | ADDRESS |
|---|---|
| Customer site | `http://localhost:5050` |
| Ops dashboard | `http://localhost:5050/ops` |
| Shelter phone | `http://localhost:5050/shelter?id=northside-youth` |
| Records | `http://localhost:5050/records` |

No internet needed for the demo itself: the map library and fonts are bundled with the app. Only Claude and ElevenLabs need a connection, and each has an offline fallback.

On a phone on the same Wi-Fi, replace `localhost` with the laptop's IP address. Campus Wi-Fi sometimes blocks this; a phone hotspot works.

### Keys and settings (in `.env`, never committed)

| SETTING | EFFECT |
|---|---|
| `ANTHROPIC_API_KEY` | Claude reads photos (and, with `LLM_AGENTS=1`, writes the agents' arguments) |
| `ELEVENLABS_API_KEY` | Real voices for the debate and shelter phone |
| `LLM_AGENTS=1` | Agents reason with Claude instead of rules |
| `AGNTCY=0` | Keep the agent processes but talk over plain HTTP instead of A2A over NATS |
| `MESH=0` | Run every agent in-process instead of as separate services |
| none | Everything runs: a labeled demo photo result, rules-based agents, the browser's voice |

Run the checks with `python tests/test_core.py` and `python tests/test_mesh.py`, and the evidence with `python scenarios.py`.

## 16 Explaining Uplink to others

### The 3-minute demo

1. **Problem (30 s).** Businesses throw away edible food, and California now requires many to donate it. But donating is a hassle full of tradeoffs.
2. **Photo (20 s).** Photograph real leftover food on the customer site. The details fill in.
3. **Agents negotiate (40 s).** On `/ops`, Speed, Cost and Need argue, the safety rules remove an unsafe shelter, and a truck starts moving.
4. **Things change (45 s).** Click "Shelter says full": they reroute. Click "Driver cancels": reliability drops and the agent that warned gains weight. Run a deadlock scenario: they ask a human. Then open Activity → Agent services and stop a shelter's agent: it drops out of the directory and the next donation goes elsewhere.
5. **Evidence (25 s).** The Evidence page: team versus single agents and the simple baseline.
6. **Close (20 s).** "Our agents negotiate real tradeoffs with no boss, safety is never up for debate, and they ask a human when they truly can't agree."

### Answers to likely questions

**Who makes the final decision?**
No single agent. A plan wins by agreement, or by majority after up to 3 rounds without a veto; otherwise a person decides.

**What if the agents never agree?**
After 3 rounds, or if the best plan crosses a red line or costs over $25, a coordinator gets the question with both sides' reasons and a deadline.

**Is the AI deciding whether food is safe?**
No. Safety is fixed rules in code (time, refrigeration, allergens, capacity, hours) that remove unsafe plans before any agent sees them.

**How do you know this beats a simple approach?**
We ran the same scenarios against a nearest-shelter baseline and against each agent alone. See section 12, including where the team costs more.

**How do you know the amount from a photo?**
The model counts what it can see and gives a range and a confidence level. Code computes servings and weight, and the business confirms or corrects every number.

**Are these really separate agents?**
Yes. Each shelter, driver and negotiator runs as its own process and is an AGNTCY A2A agent over NATS, registered in an agent directory. You can stop one live and watch the others carry on. A test proves the distributed agents decide exactly as the in-process ones.

**Is this real?**
The agents, photo reading, safety rules and records are real. The city, drivers and clock are simulated for the demo.

**How is this different from Copia?**
Copia makes businesses enter details and coordinate. Uplink cuts it to one photo, and the agents handle matching, failures, tradeoffs and paperwork.

## 17 Glossary

| TERM | MEANING |
|---|---|
| Agent | A piece of software with its own goal that makes decisions on its own |
| Agentic workflow | A process where agents act, react to events and re-plan, instead of following fixed steps once |
| Consensus | All three agents accept the same plan |
| Majority with dissent | Two of three accept; the third's objection is kept on record |
| Red line / veto | A limit one agent never accepts crossing; a majority can't override it |
| Unacceptable compromise | The majority's plan crosses the third agent's red line, so a person decides |
| Irreconcilable | After 3 rounds no plan has majority support, so a person decides |
| Insufficient evidence | The decision depends on a fact nobody has confirmed, so the agents ask before guessing |
| Safety rules | Fixed code that removes unsafe plans before negotiation |
| Fallback rule | A disclosed automatic action when nobody answers in time, so food doesn't spoil |
| Baseline | A simple comparison policy (nearest shelter, or one agent alone) used to measure the team |
| Counter-offer | A shelter offering to take only part of a donation; the rest is re-announced |
| Agent directory | A registry where agents list who they are, what they can do and where to reach them, so others can find them |
| Heartbeat | A regular "still here" signal; an agent that stops sending it drops out of the directory |
| A2A message | Agent-to-agent protocol message (used by AGNTCY); here sent over NATS, a fast message server |
| NATS | A lightweight message server; the agents' A2A messages travel through it |
| OASF record | AGNTCY's standard format for describing an agent; our directory records are modeled on it |
| Vision model | AI that understands images; here, it identifies and counts food |
| vLLM / Brev | Software that serves an open AI model; Brev rents the NVIDIA GPU it runs on |

| | |
|---|---|
| SB 1383 | California law requiring many food businesses to donate edible surplus food and keep records |
| AGNTCY | Cisco's open-source infrastructure for agent discovery and messaging (optional for this track) |